



山东大学
SHANDONG UNIVERSITY

网络安全创新创业实践课实验报告

全同态密码与应用（作业 6）

第 41 组

2026 年 7 月 30 日

小组成员及分工

本实验由小组成员共同完成，各成员主要负责内容如下：

表 1: 小组成员及分工

姓名	学号	班级	主要负责内容	对应章节
姜良萍（组长）	202300460098	网安 1 班	负责实验整体方案设计；基于作业 5 整理 Direct 密文卷积基线方法；完成实验流程设计、输入输出验证框架搭建以及最终报告整体整合。	第 1 章；第 2.1 节；第 3.1 节
王奕文	202300460056	密码 1 班	负责 BSGS 优化方案设计与理论分析；完成卷积偏移分解、Baby-Step/Giant-Step 方法分析、旋转次数理论推导、最优旋转次数证明以及理论结果整理。	第 2.2 节；第 2.3 节；第 4.3 节
李雨晴	202300460025	网安 2 班	负责 BSGS 优化算法工程实现；完成 BSGS 密文卷积函数设计、Baby-Step 旋转结果缓存、局部结果计算、Giant-Step 对齐以及操作次数统计模块实现，完成程序调试与运行测试。	第 3.2 节；第 3.3 节
曾静雯	202300460045	网安 1 班	负责实验结果验证与分析；完成 Direct 与 BSGS 方法卷积结果对比、解密槽位验证、误差分析、操作次数对比、实验截图整理以及实验总结撰写。	第 4.1 节；第 4.2 节；第 5 章

目录

1	实验目的	4
2	实验原理	4
2.1	直接旋转累加方法	4
2.2	BSGS 优化方法	6
2.3	旋转次数理论分析	8
3	实验设计与实现	11
3.1	总体设计	11
3.2	操作次数统计	13
3.3	BSGS 优化实现	15
4	结果分析	19
4.1	正确性验证	19
4.2	操作次数对比	22
4.3	理论值与实际值对比	24
5	总结	25

1 实验目的

作业 5 已基于 OpenFHE 的 CKKS 同态加密方案，完成了单输入单输出条件下 4×4 输入矩阵与 3×3 卷积核的密文卷积实现。实验采用槽打包、密文旋转、密文与明文权重乘法以及密文累加的方法完成卷积计算，并通过与明文卷积结果对比验证了实现的正确性。

本实验在作业 5 的基础上，进一步分析密文卷积中的旋转开销。首先统计直接旋转累加方法所需的密文旋转次数；随后采用 Baby-Step Giant-Step (BSGS) 策略对卷积偏移进行分组，复用中间旋转结果，以减少密文旋转操作。最后，对直接方法和 BSGS 方法的操作次数及解密结果进行比较，并分析优化后的旋转次数是否达到当前槽位布局和单层 BSGS 分组策略下的最小值。

本实验的主要目标如下：

1. 统计直接旋转累加方法中的密文旋转、密文与明文乘法及密文加法次数；
2. 设计并实现基于 BSGS 的密文卷积旋转优化方法；
3. 比较优化前后旋转次数及旋转密钥需求的变化；
4. 验证 BSGS 优化前后的密文卷积结果与明文卷积结果保持一致；
5. 分析实际旋转次数是否达到当前 BSGS 策略下的理论最小值。

2 实验原理

2.1 直接旋转累加方法

本实验沿用作业 5 中的 CKKS 槽打包方式，将 4×4 输入矩阵按照行优先顺序编码至同一个密文的多个槽位中。直接旋转累加方法的基本过程如下。

1. 确定卷积位置对应的槽位偏移

由于输入矩阵每行包含 4 个元素，卷积核位置 (r, c) 在一维槽位布局中对应的偏移量为

$$\delta_{r,c} = 4r + c, \quad r, c \in \{0, 1, 2\}.$$

因此， 3×3 卷积核的九个位置对应的槽位偏移为

$$D = \{0, 1, 2, 4, 5, 6, 8, 9, 10\}.$$

2. 对输入密文执行旋转和明文乘法

设输入矩阵经过打包和加密后得到密文 ctX ，卷积核位置 (r, c) 对应的明文权重掩码为 $W_{r,c}$ 。对于每个卷积核位置，首先将输入密文旋转 $\delta_{r,c}$ 个槽位，再与相应的明文权重掩码进行逐槽乘法：

$$ct_{r,c} = \text{Rot}_{4r+c}(ctX) \odot pt(W_{r,c}),$$

其中， $\text{Rot}_d(\cdot)$ 表示将密文槽位旋转 d 个位置， $\odot$ 表示密文与明文之间的逐槽乘法。

3. 累加各卷积位置的密文贡献

将九个卷积核位置产生的密文贡献相加，得到最终输出密文：

$$ctY = \sum_{r=0}^2 \sum_{c=0}^2 \text{Rot}_{4r+c}(ctX) \odot pt(W_{r,c}).$$

展开后可表示为

$$\begin{aligned} ctY = & ctX \odot pt(W_{0,0}) + \text{Rot}_1(ctX) \odot pt(W_{0,1}) + \text{Rot}_2(ctX) \odot pt(W_{0,2}) \\ & + \text{Rot}_4(ctX) \odot pt(W_{1,0}) + \text{Rot}_5(ctX) \odot pt(W_{1,1}) + \text{Rot}_6(ctX) \odot pt(W_{1,2}) \\ & + \text{Rot}_8(ctX) \odot pt(W_{2,0}) + \text{Rot}_9(ctX) \odot pt(W_{2,1}) + \text{Rot}_{10}(ctX) \odot pt(W_{2,2}). \end{aligned} \quad (1)$$

4. 统计直接方法的操作次数

偏移量为 0 时可以直接使用原始输入密文 ctX ，不需要执行旋转。其余 8 个非零偏移分别需要一次密文旋转，因此直接方法的旋转次数为

$$R_{\text{Direct}} = 9 - 1 = 8.$$

九个卷积核位置分别需要一次密文与明文乘法，因此

$$N_{\text{MulPlain}} = 9.$$

将九个密文贡献累加为一个输出密文需要 8 次密文加法，因此

$$N_{\text{Add}} = 9 - 1 = 8.$$

直接旋转累加方法的计算过程较为直观，但每个非零偏移都需要从原始输入密文独立生成旋转结果，没有复用已有的中间旋转结果。因此，该方法共需要 8 次密文旋转，存在进一步减少旋转开销的空间。

2.2 BSGS 优化方法

本实验采用 Baby-Step Giant-Step (BSGS) 策略，将卷积偏移分解为两级旋转，并复用中间旋转结果。具体过程如下。

1. 对卷积偏移进行 BSGS 分解

3×3 卷积核对应的九个槽位偏移为

$$D = \begin{bmatrix} 0 & 1 & 2 \\ 4 & 5 & 6 \\ 8 & 9 & 10 \end{bmatrix}.$$

其中，每个偏移均可表示为

$$\delta_{r,c} = 4r + c, \quad r, c \in \{0, 1, 2\}.$$

因此，偏移集合可分解为

$$D = \{0, 1, 2\} + \{0, 4, 8\}.$$

其中，集合 $\{0, 1, 2\}$ 作为 Baby-Step 偏移，集合 $\{0, 4, 8\}$ 作为 Giant-Step 偏移。通过这种分解，可以用少量 Baby-Step 旋转结果构造不同卷积核行对应的局部结果。

2. 生成并复用 Baby-Step 密文

设输入矩阵经过打包和加密后得到密文 ctX 。首先构造三个 Baby-Step 密文：

$$A_0 = ctX,$$

$$A_1 = \text{Rot}_1(ctX),$$

$$A_2 = \text{Rot}_2(ctX).$$

其中， A_0 直接使用原始输入密文，不需要执行旋转； A_1 和 A_2 分别需要旋转 1 个和 2 个槽位。因此，Baby-Step 阶段共需

$$R_{\text{Baby}} = 2$$

次密文旋转。

生成后的 A_0 、 A_1 和 A_2 将被卷积核的三行共同复用，从而避免分别生成 $\text{Rot}_5(ctX)$ 、 $\text{Rot}_6(ctX)$ 、 $\text{Rot}_9(ctX)$ 和 $\text{Rot}_{10}(ctX)$ 等旋转结果。

如图 1 所示，BSGS 方法首先生成少量 Baby-Step 旋转结果，并将其复用于三个卷积核行的局部计算。相比于直接方法分别生成每个偏移对应的旋转密文，BSGS 方法避免了重复旋转操作。

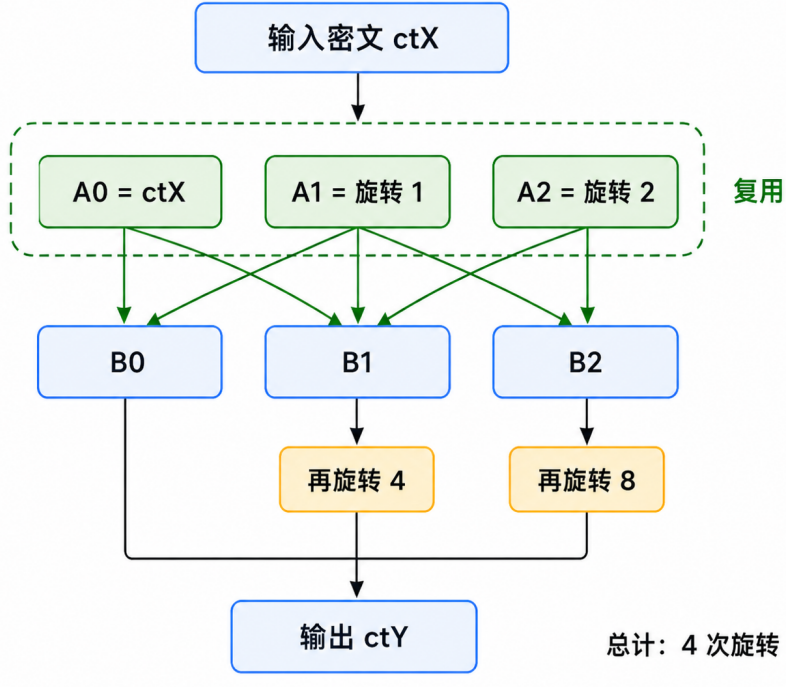


图 1: BSGS 方法中 Baby-Step 旋转结果复用过程

3. 构造各卷积核行的局部结果

对卷积核第 r 行，利用三个 Baby-Step 密文计算对应的局部结果：

$$B_r = \sum_{c=0}^2 A_c \odot pt(\widetilde{W}_{r,c}), \quad r \in \{0, 1, 2\},$$

其中， $\widetilde{W}_{r,c}$ 表示经过预移位处理的明文权重掩码。

由于局部结果 B_1 和 B_2 在后续 Giant-Step 阶段还需要分别左旋 4 个和 8 个槽位，因此对应的明文掩码需要预先向相反方向移动。设直接方法中的明文权重掩码为 $W_{r,c}$ ，则预移位后的掩码为

$$\widetilde{W}_{r,c} = \text{Rot}_{-4r}(W_{r,c}).$$

因此，不同卷积核行的掩码调整方式为

$$\begin{cases} r = 0: & \text{掩码不移动,} \\ r = 1: & \text{掩码向右预移 4 个槽位,} \\ r = 2: & \text{掩码向右预移 8 个槽位.} \end{cases}$$

通过提前调整明文掩码，局部结果在执行后续 Giant-Step 旋转后，能够重新对齐至有效输出槽位 0、1、4 和 5。

4. 执行 Giant-Step 旋转并累加

得到三个局部结果后，分别根据其所在卷积核行执行 Giant-Step 旋转，最终输出密文可表示为

$$ctY = B_0 + \text{Rot}_4(B_1) + \text{Rot}_8(B_2).$$

其中， B_0 不需要旋转， B_1 和 B_2 分别需要旋转 4 个和 8 个槽位。因此，Giant-Step 阶段共需

$$R_{\text{Giant}} = 2$$

次密文旋转。

5. 统计 BSGS 方法的操作次数

BSGS 方法的总旋转次数为

$$R_{\text{BSGS}} = R_{\text{Baby}} + R_{\text{Giant}} = 2 + 2 = 4.$$

虽然 BSGS 改变了密文旋转与累加的组织方式，但每个卷积核权重仍需参与一次密文与明文乘法，因此

$$N_{\text{MulPlain}} = 9.$$

九个卷积贡献最终仍需合并为一个输出密文，因此密文加法次数保持为

$$N_{\text{Add}} = 8.$$

综上，BSGS 方法通过复用偏移 0、1 和 2 对应的 Baby-Step 密文，将九个卷积偏移重组为三个卷积核行的局部结果，再通过偏移 0、4 和 8 完成 Giant-Step 对齐。该方法将密文旋转次数由直接方法的 8 次减少为 4 次，同时保持密文与明文乘法次数和密文加法次数不变。

2.3 旋转次数理论分析

为了分析 BSGS 方法的旋转次数是否达到理论最优，本实验进一步对 Baby-Step Giant-Step 策略下的旋转开销进行理论分析。

设卷积计算中需要处理的偏移数量为 n ，Baby-Step 的分组大小为 b 。对于本实验的 3×3 卷积核，共包含 9 个槽位偏移：

$$n = 9.$$

2.3.1 BSGS 旋转代价模型

在 BSGS 方法中，所有偏移被划分为若干个大小为 b 的分组。旋转次数主要由 Baby-Step 和 Giant-Step 两部分组成。

(1) Baby-Step 旋转次数

Baby-Step 阶段需要生成

$$0, 1, \dots, b-1$$

对应的旋转结果。其中，偏移 0 可以直接使用原始密文，不需要执行旋转。因此 Baby-Step 阶段的旋转次数为：

$$R_{\text{Baby}} = b - 1.$$

(2) Giant-Step 旋转次数

设总共需要划分的分组数量为：

$$g = \left\lceil \frac{n}{b} \right\rceil.$$

第一个分组无需执行 Giant-Step 旋转，其余每个分组需要一次整体偏移。因此 Giant-Step 阶段的旋转次数为：

$$R_{\text{Giant}} = \left\lceil \frac{n}{b} \right\rceil - 1.$$

因此，BSGS 方法总旋转次数可表示为：

$$R(b) = R_{\text{Baby}} + R_{\text{Giant}} = (b - 1) + \left(\left\lceil \frac{n}{b} \right\rceil - 1 \right).$$

对于本实验中的 9 个卷积偏移，有：

$$R(b) = (b - 1) + \left(\left\lceil \frac{9}{b} \right\rceil - 1 \right)$$

2.3.2 最优 Baby-Step 大小分析

为了寻找最优的 Baby-Step 分组大小，对不同 b 值对应的旋转次数进行计算，结果如下：

表 2: 不同 Baby-Step 大小下的 BSGS 旋转次数

Baby-Step 大小 b	旋转次数 $R(b)$
1	8
2	5
3	4
4	5
5	5
6	6
7	7
8	8
9	8

由表可知，当 Baby-Step 大小取：

$$b = 3$$

时，旋转次数达到最小值：

$$R(3) = (3 - 1) + \left(\left\lceil \frac{9}{3} \right\rceil - 1 \right) = 2 + 2 = 4.$$

因此，在本实验采用的 BSGS 分组策略下，理论最小旋转次数为：

$$R_{\min} = 4.$$

2.3.3 理论值与实际实现对比

本实验实现的 BSGS 方法中：

$$\text{Rot}_1(ctX), \text{Rot}_2(ctX)$$

用于生成 Baby-Step 结果，因此：

$$R_{\text{Baby}} = 2.$$

随后，通过：

$$\text{Rot}_4(B_1), \text{Rot}_8(B_2)$$

完成 Giant-Step 对齐，因此：

$$R_{\text{Giant}} = 2.$$

最终实际旋转次数为：

$$R_{\text{actual}} = R_{\text{Baby}} + R_{\text{Giant}} = 2 + 2 = 4.$$

由于：

$$R_{\text{actual}} = R_{\text{min}} = 4,$$

因此说明本实验实现的 BSGS 方法达到了当前槽位布局和单层 BSGS 分组策略下的最小旋转次数。

3 实验设计与实现

3.1 总体设计

本实验在作业 5 实现的 CKKS 密文卷积程序基础上进行扩展,保持输入数据、CKKS 参数配置以及正确性验证方式不变,重点针对密文卷积过程中的旋转操作进行优化。

实验程序同时实现两种密文卷积计算路径：直接旋转累加方法 (Direct) 和基于 Baby-Step Giant-Step (BSGS) 的优化方法。两种方法均以同一个输入密文 ctX 作为计算起点,分别执行对应的密文卷积计算,得到输出密文：

$$ctY_{\text{Direct}}, \quad ctY_{\text{BSGS}}.$$

随后,对两种方法得到的输出密文分别进行解密,提取有效输出槽位中的结果,并与明文卷积参考结果进行比较。同时,程序自动统计两种方法执行过程中的密文旋转、密文与明文乘法以及密文加法次数,用于分析 BSGS 方法的优化效果。

实验整体流程如图 2 所示。

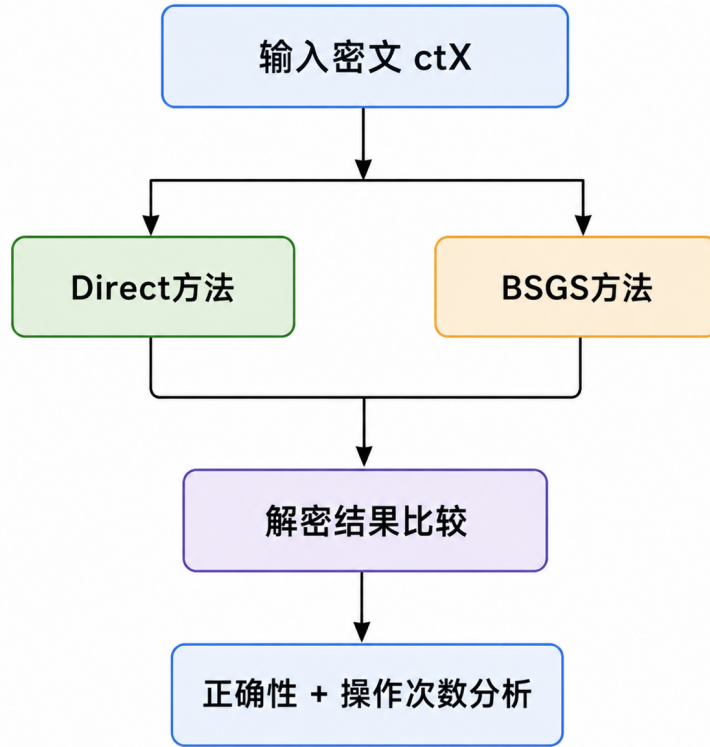


图 2: 密文卷积旋转优化实验总体流程

具体实验流程如下:

1. 输入密文生成

首先沿用作业 5 中的 CKKS 槽打包方案, 将输入矩阵编码并加密, 得到输入密文 ctX 。该密文作为两种卷积计算方法的共同输入。

2. 执行两种密文卷积方法

输入密文 ctX 分别进入两条计算路径:

- **Direct 方法:** 根据卷积核偏移逐个执行密文旋转、密文与明文权重乘法以及密文累加;
- **BSGS 方法:** 利用 Baby-Step Giant-Step 策略对卷积偏移进行分组, 复用中间旋转结果, 减少密文旋转次数。

3. 结果解密与比较

两种方法计算完成后, 分别对输出密文进行解密, 并提取有效输出槽位中的卷积结果。随后与明文卷积结果进行比较, 通过最大绝对误差判断计算结果是否正确。

4. 操作次数统计与分析

程序在执行过程中自动记录密文旋转、密文与明文乘法以及密文加法次数，并对 Direct 方法和 BSGS 方法的计算开销进行比较。

为了便于实验实现和结果分析，程序按照功能划分为多个模块，各模块功能如下表所示。

表 3: 实验程序模块划分

模块	功能
PlainConv2D	实现明文二维卷积计算，为密文卷积结果提供参考值，用于后续正确性验证。
EncryptedConv2DDirect	实现直接旋转累加方法，根据卷积核偏移分别执行密文旋转、密文与明文权重乘法以及密文累加。
EncryptedConv2DBSGS	实现基于 BSGS 的旋转优化方法，通过 Baby-Step 和 Giant-Step 分解卷积偏移，并复用中间旋转结果减少旋转次数。
OperationStats	记录实验过程中密文旋转、密文与明文乘法以及密文加法次数，用于定量比较不同方法的计算开销。
DecryptSlots	完成输出密文解密，并从 CKKS 槽位中提取有效卷积输出结果。
MaxAbsoluteError	计算密文卷积结果与明文参考结果之间的最大绝对误差，用于判断计算正确性。

3.2 操作次数统计

为了定量分析 BSGS 方法对旋转操作的优化效果，本实验在程序中增加操作次数统计模块，对 Direct 方法和 BSGS 方法执行过程中的主要同态操作进行自动记录。

3.2.1 统计指标

本实验主要统计以下三类同态操作：

1. 密文旋转次数

密文旋转用于实现卷积窗口中不同位置输入元素的对齐，对应 OpenFHE 中的 `EvalRotate()` 操作。实验中使用

$$N_{\text{Rot}}$$

表示密文旋转次数。

对于 Direct 方法，该指标反映九个卷积偏移分别生成旋转密文的开销；对于 BSGS 方法，该指标反映通过旋转结果复用后剩余的旋转开销。

2. 密文与明文乘法次数

密文与明文权重向量的逐槽乘法用于完成卷积核权重计算，对应 OpenFHE 中的 `EvalMult(ciphertext, plaintext)` 操作。实验使用

$$N_{\text{MulPlain}}$$

表示密文与明文乘法次数。

由于 3×3 卷积核包含 9 个权重位置，两种方法均需要对九个卷积核权重进行计算，因此理论上均满足：

$$N_{\text{MulPlain}} = 9.$$

3. 密文加法次数

不同卷积位置产生的密文贡献需要进行累加，对应 OpenFHE 中的 `EvalAdd()` 操作。实验使用

$$N_{\text{Add}}$$

表示密文加法次数。

九个卷积贡献最终需要合并为一个输出密文，因此两种方法均需要：

$$N_{\text{Add}} = 9 - 1 = 8.$$

3.2.2 统计模块实现

为了避免根据算法流程进行人工统计，本实验在程序中设计了操作次数统计结构 `OperationStats`，用于保存当前密文卷积过程中的操作数量：

$$\text{OperationStats} = (N_{\text{Rot}}, N_{\text{MulPlain}}, N_{\text{Add}}).$$

在具体实现中，实验对 OpenFHE 基础同态操作进行了封装，分别定义：

- `CountedRotate()`：执行密文旋转，同时更新 N_{Rot} ；
- `CountedMultPlain()`：执行密文与明文乘法，同时更新 N_{MulPlain} ；

- `CountedAdd()`: 执行密文加法, 同时更新 N_{Add} 。

例如, 每执行一次密文旋转操作:

$$N_{\text{Rot}} \leftarrow N_{\text{Rot}} + 1.$$

通过上述方式, 程序运行结束后可以直接获得两种密文卷积方法的实际操作数量, 避免由于手动统计导致的误差。

3.2.3 操作次数比较

实验分别运行直接旋转累加函数 `EncryptedConv2DDirect()` 和 BSGS 优化函数 `EncryptedConv2DBSGS()`, 并记录两种方法对应的:

$$(N_{\text{Rot}}, N_{\text{MulPlain}}, N_{\text{Add}})$$

作为计算开销评价指标。

其中, 密文旋转次数是本实验重点关注的优化对象, 通过比较:

$$N_{\text{Rot}}^{\text{Direct}} \quad \text{和} \quad N_{\text{Rot}}^{\text{BSGS}}$$

分析 BSGS 方法利用中间旋转结果复用带来的性能优化效果。

3.3 BSGS 优化实现

在程序实现方面, 本实验在作业 5 直接旋转累加方法的基础上, 新增 `EncryptedConv2DBSGS()` 函数, 用于实现基于 BSGS 的旋转优化卷积。该实现保持输入密文、卷积核参数以及输出槽位布局不变, 仅调整密文旋转的组织方式, 通过复用中间旋转结果减少重复计算。

3.3.1 函数结构

BSGS 优化主要由两个函数完成:

- `EncryptedConv2DBSGS()`: 负责整体 BSGS 卷积流程, 包括 Baby-Step 密文生成、局部结果计算、Giant-Step 旋转以及最终密文累加;
- `ComputeKernelRowBSGS()`: 负责计算单个卷积核行对应的局部贡献, 复用已经生成的 Baby-Step 密文结果。

整体程序执行流程如图 3 所示。

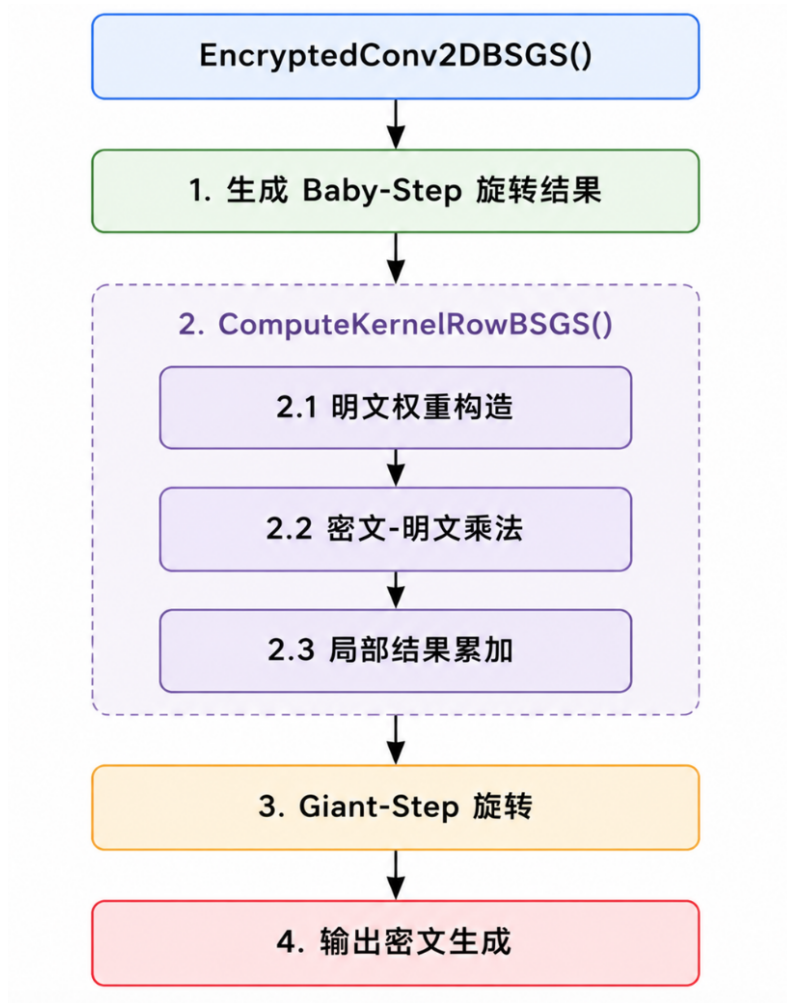


图 3: BSGS 密文卷积实现流程

3.3.2 Baby-Step 结果缓存

程序首先生成并保存 Baby-Step 阶段的中间密文结果：

Listing 1: Baby-Step 旋转结果生成

```

1 std::array<Ciphertext<DCRTPoly>, KERNEL_W> babyRotations{
2     encryptedInput,
3     Rot1(ctX),
4     Rot2(ctX)
5 };
  
```

其中：

$$A_0 = ctX,$$

$$A_1 = \text{Rot}_1(ctX),$$

$$A_2 = \text{Rot}_2(ctX).$$

生成后的三个密文结果会被三个卷积核行共同复用，而不需要针对每个卷积偏移重新执行旋转操作。

相比直接方法中分别计算：

$$\text{Rot}_5(ctX), \text{Rot}_6(ctX), \text{Rot}_9(ctX), \text{Rot}_{10}(ctX),$$

BSGS 方法避免了重复的旋转计算。

3.3.3 局部结果计算与掩码处理

在获得 Baby-Step 中间结果后，程序调用 `ComputeKernelRowBSGS()` 函数计算每一行卷积核对应的局部贡献。

函数内部主要完成以下步骤：

1. 根据当前卷积核位置获取对应权重；
2. 构造 CKKS 明文权重向量；
3. 执行密文与明文逐槽乘法；
4. 累加得到当前卷积核行结果。

核心计算过程如下：

Listing 2: 局部卷积结果计算

```

1 auto contribution =
2     CountedMultPlain(
3         cryptoContext,
4         babyRotation,
5         weightPlaintext,
6         stats);
7
8 rowSum =
9     CountedAdd(
10        cryptoContext,
11        rowSum,
12        contribution,
13        stats);

```

由于不同卷积核行在后续需要执行不同的 Giant-Step 旋转，程序在生成明文权重掩码时进行了预移位处理，使局部结果经过后续旋转后能够重新对齐到有效输出槽位。

3.3.4 输出密文生成

完成三个卷积核行局部结果计算后，程序执行 Giant-Step 旋转：

Listing 3: Giant-Step 旋转与结果对齐

```
1 auto shiftedRow1 =  
2     CountedRotate(  
3         cryptoContext,  
4         rowSums[1],  
5         4,  
6         stats);  
7  
8 auto shiftedRow2 =  
9     CountedRotate(  
10        cryptoContext,  
11        rowSums[2],  
12        8,  
13        stats);
```

对应数学形式为：

$$B'_1 = \text{Rot}_4(B_1),$$

$$B'_2 = \text{Rot}_8(B_2).$$

随后将三个部分结果累加，得到最终输出密文：

$$ctY = B_0 + \text{Rot}_4(B_1) + \text{Rot}_8(B_2).$$

对应代码实现为：

Listing 4: 输出密文累加

```
1 auto encryptedOutput =  
2     CountedAdd(  
3         cryptoContext,  
4         rowSums[0],  
5         shiftedRow1,  
6         stats);  
7  
8 encryptedOutput =  
9     CountedAdd(  
10        cryptoContext,
```

```
11     encryptedOutput ,  
12     shiftedRow2 ,  
13     stats);
```

通过上述实现，BSGS 方法在保持原有卷积计算语义的同时，复用了中间旋转结果，并将密文旋转次数由直接方法的 8 次降低至 4 次。

4 结果分析

4.1 正确性验证

为了验证 BSGS 优化方法在减少旋转次数的同时是否保持原有密文卷积计算结果，本实验分别对 Direct 方法和 BSGS 方法得到的输出密文进行解密，并与明文卷积参考结果进行比较。正确性验证主要包括卷积结果对比、输出槽位布局验证以及误差分析三个部分。

4.1.1 卷积结果对比

实验首先比较明文卷积结果、Direct 方法解密结果以及 BSGS 方法解密结果。实验结果如图 4 所示。

```
[3] Convolution results  
  
Plaintext reference =  
[ 12.230000 13.270000  
 18.640000 19.950000 ]  
  
Direct FHE result =  
[ 12.230000 13.270000  
 18.640000 19.950000 ]  
  
BSGS FHE result =  
[ 12.230000 13.270000  
 18.640000 19.950000 ]
```

图 4: Direct 与 BSGS 方法卷积结果对比

由图 4 可知，三种方法得到的输出矩阵均为：

$$Y = \begin{bmatrix} 12.23 & 13.27 \\ 18.64 & 19.95 \end{bmatrix}.$$

其中，Direct 方法采用原始旋转累加方式计算，BSGS 方法通过复用中间旋转结果减少旋转次数，但两者最终得到的卷积输出完全一致。

实验结果表明，BSGS 方法仅改变了密文旋转和累加的组织方式，并未改变卷积计算语义。

4.1.2 输出槽位验证

为了进一步验证 BSGS 方法是否保持原有 CKKS 输出布局，实验输出两种方法解密后的完整槽位结果，如图 5 所示。

```

[4] Decrypted slot layouts

Direct decrypted slots
[  12.230000  13.270000  0.000000  0.000000
   18.640000  19.950000  0.000000  0.000000
    0.000000  0.000000  0.000000  0.000000
    0.000000  0.000000  0.000000  0.000000 ]

BSGS decrypted slots
[  12.230000  13.270000  0.000000  0.000000
   18.640000  19.950000  0.000000  0.000000
    0.000000  0.000000  0.000000  0.000000
    0.000000  0.000000  0.000000  0.000000 ]

```

图 5: Direct 与 BSGS 方法解密后的槽位布局

根据作业 5 中的槽位设计，卷积输出矩阵对应的有效槽位为：

$$\{0, 1, 4, 5\}.$$

由图 5 可知，两种方法解密后的槽位布局均满足：

$$slot[0] = y_{00},$$

$$slot[1] = y_{01},$$

$$slot[4] = y_{10},$$

$$slot[5] = y_{11}.$$

其余槽位均接近于 0。

因此，BSGS 方法虽然改变了旋转计算过程，但没有改变输出槽位布局，说明前述明文掩码调整以及 Giant-Step 对齐过程正确。

4.1.3 误差分析

由于 CKKS 属于近似同态加密方案，密文计算结果与明文计算结果之间允许存在微小数值误差。因此，本实验采用最大绝对误差作为正确性评价指标：

$$e_{\max} = \max_i |y_i^{FHE} - y_i^{plain}|.$$

实验正确性验证结果如图 6 所示。

```
[6] Correctness verification
Direct maximum error      : 2.22044605e-13
BSGS maximum error        : 3.65929509e-13
Difference of methods      : 2.13162821e-13
Error tolerance           : 1.00000000e-05
Rotation reduction         : 8 -> 4 (50%)
Verification               : PASS
```

图 6: Direct 与 BSGS 方法正确性验证结果

实验测得最大绝对误差如下：

$$e_{\text{Direct}} = 2.22044605 \times 10^{-13},$$

$$e_{\text{BSGS}} = 3.65929509 \times 10^{-13}.$$

同时，两种密文卷积方法之间的最大差异为：

$$e_{\text{diff}} = 2.13162821 \times 10^{-13}.$$

实验设置的误差阈值为：

$$\epsilon = 10^{-5}.$$

由于：

$$e_{\text{Direct}} < \epsilon,$$

$$e_{\text{BSGS}} < \epsilon,$$

因此两种方法均满足精度要求，程序输出：

Verification = *PASS*.

实验结果表明，BSGS 优化方法在减少密文旋转次数的同时，没有引入额外的计算误差，能够正确完成密文卷积计算。

4.2 操作次数对比

为了定量分析 BSGS 方法对密文卷积计算开销的优化效果，实验分别运行 Direct 方法和 BSGS 方法，并统计两种方法执行过程中的主要同态操作数量。统计指标包括密文旋转次数、密文与明文乘法次数以及密文加法次数。

4.2.1 同态操作数量对比

实验统计结果如图 7 所示。

[5] Operation-count comparison

Operation	Direct	BSGS
Rotations	8	4
Plain multiplications	9	9
Ciphertext additions	8	8

Direct rotation keys : {1, 2, 4, 5, 6, 8, 9, 10}
 BSGS rotation keys : {1, 2, 4, 8}

图 7: Direct 与 BSGS 方法操作次数对比

根据实验统计结果，整理得到两种方法的操作次数如下表所示。

表 4: Direct 与 BSGS 方法同态操作次数对比

操作类型	Direct 方法	BSGS 方法
密文旋转次数	8	4
密文与明文乘法次数	9	9
密文加法次数	8	8

4.2.2 旋转次数优化分析

由表中结果可知，BSGS 方法主要减少了密文旋转操作。

直接旋转累加方法需要针对 8 个非零卷积偏移分别执行密文旋转，因此：

$$N_{\text{Rot}}^{\text{Direct}} = 8.$$

而 BSGS 方法通过复用 Baby-Step 阶段生成的中间旋转结果，仅需要执行：

$$N_{\text{Rot}}^{\text{BSGS}} = 4.$$

因此旋转次数减少比例为：

$$\frac{N_{\text{Rot}}^{\text{Direct}} - N_{\text{Rot}}^{\text{BSGS}}}{N_{\text{Rot}}^{\text{Direct}}} \times 100\% = \frac{8 - 4}{8} \times 100\% = 50\%.$$

实验结果表明，BSGS 方法能够有效降低密文旋转开销。

同时，两种方法的密文与明文乘法次数均为：

$$N_{\text{MulPlain}} = 9,$$

密文加法次数均为：

$$N_{\text{Add}} = 8.$$

说明 BSGS 优化并未减少卷积计算本身的乘法和累加过程，而是通过重新组织旋转过程，减少了重复的槽位旋转操作。

4.2.3 旋转密钥需求分析

除旋转次数外，本实验进一步比较两种方法所需的旋转密钥集合。

Direct 方法需要支持以下旋转：

$$K_{\text{Direct}} = \{1, 2, 4, 5, 6, 8, 9, 10\}.$$

共需要 8 个旋转密钥。

而 BSGS 方法仅需要：

$$K_{\text{BSGS}} = \{1, 2, 4, 8\}.$$

共需要 4 个旋转密钥。

因此，BSGS 方法不仅减少了运行过程中的密文旋转次数，同时降低了所需旋转密钥的数量，有助于减少旋转相关的密钥存储和管理开销。

综上，实验结果表明，BSGS 方法在保持密文与明文乘法次数以及密文加法次数不变的情况下，将密文旋转次数由 8 次降低至 4 次，实现了 50% 的旋转开销降低。

4.3 理论值与实际值对比

根据前文对 BSGS 旋转次数的理论分析，当 Baby-Step 大小取 $b = 3$ 时，旋转次数达到最小值：

$$R(3) = 4.$$

为了验证理论分析与实际实现的一致性，实验程序在运行过程中自动计算 BSGS 策略下的理论最小旋转次数，并统计实际执行的密文旋转次数。实验输出结果如图 8 所示。

```
BSGS cost formula      : R(b) = (b - 1) + (ceil(9 / b) - 1)
Optimal baby-step size : b = 3
BSGS minimum rotations : R(3) = 4
Actual BSGS rotations  : 4
Minimum reached        : YES
```

图 8: BSGS 理论旋转次数与实际旋转次数对比

由图 8 可知：

$$R_{\min} = 4,$$

而程序实际执行：

$$R_{\text{actual}} = 4.$$

因此：

$$R_{\text{actual}} = R_{\min}.$$

实验结果表明，所实现的 BSGS 密文卷积方法达到了当前槽位布局和单层 BSGS 分组策略下的理论最小旋转次数。

5 总结

本实验在作业 5 密文卷积实现的基础上，引入 BSGS 优化方法，将密文旋转次数由 8 次降低至 4 次，同时保持计算结果正确，达到了当前 BSGS 分组策略下的理论最优值。

- **实验理解**

通过本实验认识到，同态加密中的性能优化不仅需要关注单个运算的效率，还需要结合数据布局和计算流程进行优化。在密文卷积中，旋转操作会带来较大的计算开销，因此减少不必要的旋转是提升效率的重要方向。

- **方法认识**

BSGS 方法通过分析卷积偏移之间的关系，将部分旋转结果进行复用，避免了直接方法中重复生成旋转密文的问题。该实验说明，合理组织计算过程可以在不改变计算结果的情况下减少同态操作数量。

- **后续思考**

本实验主要针对单输入单输出的小规模卷积进行了优化。对于更复杂的卷积计算，还可以进一步探索多通道场景下的旋转复用、数据编码优化以及底层加速方法。